Distributed Systems Patterns

Applied to 50 Real-World Systems

Where and why each pattern from Unmesh Joshi's book is used — for every system you are building

How to Use This Guide

This guide maps all 29 patterns from Patterns of Distributed Systems to 50 specific systems you are building. For each system, you will find a table showing: the specific component or layer inside the system, which pattern(s) apply there, and the precise reason that pattern is the right choice for that component.

Each system also includes a Key Insight — the one pattern that is most architecturally defining for that system. When you sit down to design any of these systems, start there.

Master Pattern Cross-Reference

The table below shows which patterns appear most frequently across systems. Patterns at the top are the most universally applicable.

Pattern	Appears In	Category	Why So Common
Write-Ahead Log	44 / 50 systems	Durability	The universal write-durability primitive. Any system that stores state needs it.
Replicated Log	38 / 50 systems	Replication	Every replicated system is built on this. The log IS the truth.
Leader and Followers	35 / 50 systems	Replication	The dominant replication model for strongly-consistent systems.
Idempotent Receiver	34 / 50 systems	Client Patterns	Any system with retries needs this. Networks are unreliable. Period.
Segmented Log	32 / 50 systems	Durability	WALs always grow unbounded without segmentation. Essential companion to WAL.
Follower Reads	30 / 50 systems	Replication	The primary horizontal read-scale mechanism in replicated systems.
Consistent Core	28 / 50 systems	Cluster Mgmt	Coordination always needs a small, strongly-consistent subsystem.
Versioned Value	28 / 50 systems	Distributed Time	MVCC is everywhere. Historical reads, conflict detection, cache validation.

Lease	26 / 50 systems	Cluster Mgmt	The fencing mechanism. Prevents double-work, double-booking, double-leadership.
State Watch	24 / 50 systems	Cluster Mgmt	Push > poll. Used whenever a component must react to changes in another.
Majority Quorum	20 / 50 systems	Consensus	The mathematical guarantee against split-brain. Required wherever consistency is critical.
HeartBeat	20 / 50 systems	Failure Detect.	Every distributed system must detect node failures. This is how.
Fixed Partitions	18 / 50 systems	Partitioning	Stable data sharding without rehashing. Kafka, Redis, every large data store.
Key-Range Partitions	16 / 50 systems	Partitioning	Required when range queries matter. Time-series, geo, sorted data.
Gossip Dissemination	16 / 50 systems	Cluster Mgmt	Decentralized metadata propagation. Scales where broadcasting doesn't.
Hybrid Clock	14 / 50 systems	Distributed Time	Real-time ordering without a central clock. Used in modern DBs and ID generators.
Singular Update Queue	14 / 50 systems	Communication	Serialize access to any shared state component. The actor model's core mechanism.
Generation Clock	12 / 50 systems	Consensus	Reject stale leaders. Required in any system that elects leaders.
Sharded Counter	10 / 50 systems	Infrastructure	High-write counters (likes, views, clicks) at scale.
Two-Phase Commit	8 / 50 systems	Partitioning	Atomic cross-node transactions. Reserved for financial/inventory systems.
Version Vector	8 / 50 systems	Distributed Time	Conflict detection in leaderless/multi-master systems. Dynamo, CRDTs.
High-Water Mark	8 / 50 systems	Replication	The safe read boundary in leader-follower replication.
Lamport Clock	6 / 50 systems	Distributed Time	Causal ordering without wall clocks. Foundational, often superseded by HLC.
Low-Water Mark	8 / 50 systems	Durability	The log compaction boundary. Always paired with Segmented Log.
Request Batch	8 / 50 systems	Communication	Throughput optimization. Critical on any high-volume write path.
Request Pipeline	6 / 50 systems	Communication	Latency optimization for sequential request streams.
Single-Socket Channel	8 / 50 systems	Communication	Message ordering between two nodes. WebSocket, Kafka broker connections.
Request Waiting List	6 / 50 systems	Communication	Async coordination for quorum responses and cache-miss coalescing.
Clock-Bound Wait	4 / 50 systems	Distributed Time	Financial/regulatory timestamp accuracy. Google Spanner, stock exchanges.
Paxos	2 / 50 systems	Consensus	The consensus proof. Used directly only in Google-scale infrastructure.
Emergent Leader	4 / 50 systems	Cluster Mgmt	Age-based leadership. Kafka KRaft, ZooKeeper internals.

Systems Index

Part 1 — Infrastructure & Building Blocks

1. DNS (Domain Name System)
2. Load Balancer
3. Key-Value Store
4. CDN (Content Delivery Network)
5. Sequencer (ID Generator)
6. Distributed Monitoring
7. Monitor Server Side Errors
8. Monitor Client Side Errors
9. Distributed Cache
10. Distributed Messaging Queue
11. Pub-Sub System
12. Rate Limiter
13. Blob Store
14. Distributed Search
15. Distributed Logging
16. Distributed Task Scheduler
17. Sharded Counter

Part 2 — Product Systems

1. YouTube / TikTok (Video Platform)
2. Ticketmaster (Event Ticketing)
3. DoorDash / Uber Eats (Food Delivery)
4. Quora / Reddit (Q&A / Forum)
5. Google Maps (Mapping Platform)
6. Discord (Real-Time Chat)
7. Proximity Service / Nearby Friends
8. Uber (Ride Hailing)
9. Twitter / X (Microblogging)
10. Newsfeed System
11. Instagram
12. URL Shortening Service
13. Web Crawler
14. WhatsApp / Facebook Messenger
15. Typeahead Suggestion (Search Autocomplete)
16. Google Docs (Collaborative Editing)
17. Deployment System (CI/CD)
18. Payment System
19. ChatGPT / AI Chat System
20. Dropbox / Google Drive (File Sync)
21. Stock Exchange
22. Distributed Email Service
23. Real-Time Gaming Leaderboard
24. Ad Click Event Aggregation

25. Hotel Reservation System
26. Uber Eats (Food Delivery Platform)
27. Facebook Messenger (Messaging Platform)
28. Data Infrastructure System (Data Platform)
29. Chat System (Generic / Enterprise)
30. Nearby Friends
31. Google Drive
32. Digital Wallet (e.g., PayPal, Venmo, Apple Pay)

DNS (Domain Name System)

Infrastructure

OVERVIEW	DNS maps domain names to IPs. It is the world's largest distributed key-value cache with hierarchical authority delegation. Every pattern here deals with consistency and propagation across thousands of resolvers.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Zone data storage	Write-Ahead Log	Every zone file change is logged before committed so zone servers can recover after a crash without serving stale/corrupt records
TTL-based cache expiry	Versioned Value	Each DNS record carries a version (serial number in SOA). Resolvers cache the versioned value and revalidate after TTL; versioning detects stale cache entries
Record propagation	Gossip Dissemination	Secondary nameservers learn of zone changes via NOTIFY + AXFR/IXFR — functionally identical to gossip: push change to a few, they pull and propagate
Authoritative cluster	Leader and Followers + Replicated Log	Primary nameserver is the leader; secondaries are followers that replicate the zone log. IXFR (incremental zone transfer) IS the replicated log
Resolver cluster health	HeartBeat + Consistent Core	Anycast resolver clusters heartbeat each other; a consistent core (like etcd) tracks which resolver nodes are healthy for routing
Change coordination	Lease + State Watch	When a zone update is in flight, a lease prevents two operators from editing the same zone simultaneously. State watches notify secondaries to re-transfer

KEY INSIGHT	DNS is fundamentally a globally replicated key-value store with hierarchical partitioning (zones) and tunable consistency (TTL). The Replicated Log + Leader/Followers pattern explains primary→secondary replication exactly.

Load Balancer

Infrastructure

OVERVIEW	Distributes incoming traffic across a pool of backend servers. Must detect failures instantly, maintain session affinity, and handle millions of connections with minimal latency.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Backend health tracking	HeartBeat	Load balancer sends health probes (HTTP /health or TCP SYN) every N seconds. If a backend misses K consecutive beats, it's removed from rotation — textbook HeartBeat
Session affinity / sticky sessions	Lease	A client-to-backend mapping is granted as a time-bounded lease (e.g., 30 min). If the backend dies, the lease expires and the client is rehashed to a new backend
Backend pool membership	Gossip Dissemination	In L7 mesh load balancers (Envoy, Linkerd), backend availability is gossiped across all LB instances rather than depending on a central registry
Config distribution	Replicated Log + State Watch	When an operator adds a backend, a replicated log ensures all LB instances see the same config. State watches trigger hot reload with zero downtime
Request routing state	Consistent Core	For consistent hashing ring membership and backend weight changes, a small consistent core (etcd/ZooKeeper) holds the authoritative ring state
Connection queuing	Singular Update Queue	Each backend's connection queue is managed by a single-threaded dispatcher to avoid race conditions on connection counts and backpressure signals

KEY INSIGHT	The load balancer itself is stateless for requests but stateful for membership. HeartBeat + Lease handles failure detection and session stickiness; Consistent Core handles the authoritative backend pool.

Key-Value Store

Infrastructure

OVERVIEW	The foundational distributed storage primitive. The pattern choices here literally define whether you're building something like Redis, etcd, Dynamo, or Cassandra.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Write path	Write-Ahead Log → Replicated Log	Every PUT is appended to the WAL before ACK. The WAL is then replicated to followers. This is exactly how etcd, TiKV, and RocksDB-based stores work
Read path (strong)	Leader and Followers + High-Water Mark	Strong reads go to the leader; leader only returns entries up to the high-water mark (committed index) — prevents reading uncommitted data
Read path (eventual)	Follower Reads + Versioned Value	Eventual-consistency reads go to any replica. Versioned values let clients detect and reject stale responses (read your own writes with version check)
Storage layout	Segmented Log + Low-Water Mark	On-disk SST files are segments. Compaction uses the low-water mark to know which old segments are safe to merge/delete
Concurrency control	Versioned Value (MVCC)	Each key stores multiple timestamped versions. Readers get a snapshot at a timestamp; writers create new versions. No read-write lock needed
Conflict detection (leaderless)	Version Vector	In Dynamo-style stores, each value carries a version vector. On concurrent writes, the store detects conflicts and either auto-merges or surfaces them to the client
Ordering across nodes	Hybrid Clock	HLC timestamps attached to each value give causally-consistent ordering across nodes without requiring a central clock
Partitioning	Fixed Partitions or Key-Range Partitions	Fixed (hash-based) for uniform distribution (Redis Cluster, Dynamo); Key-Range for range query support (Bigtable, CockroachDB, TiKV)
Leader election	Generation Clock + Majority Quorum	When a leader dies, new election increments the generation clock. A node wins only with a quorum majority, preventing split-brain
Client retries	Idempotent Receiver	Clients assign request IDs. On timeout+retry, the server deduplicates — prevents double-writes on network failures

KEY INSIGHT	A KV store is the most pattern-dense system. The axis of choice: strong (WAL + Replicated Log + Leader/Followers + Quorum) vs. eventual (Version Vector + Follower Reads + Gossip). You cannot build either correctly without these patterns.

CDN (Content Delivery Network)

Infrastructure

OVERVIEW	Caches content at edge PoPs geographically close to users. Reduces origin load and latency. The main challenges are cache invalidation, origin pull coordination, and PoP health.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Edge cache invalidation	Versioned Value + State Watch	Every cached object has a version/ETag. When origin purges content, State Watch notifies all edge nodes to drop that version. Cache-Control + ETags IS the Versioned Value pattern
PoP health monitoring	HeartBeat + Gossip Dissemination	Each PoP heartbeats a regional controller. PoP-to-PoP availability gossips across the network so DNS can route around failures within seconds
Origin shield coordination	Consistent Core + Lease	Only one edge node in a cluster should be the origin-puller (shield). A lease grants this role; on expiry, another node takes over — prevents thundering herd
Request coalescing	Request Waiting List + Singular Update Queue	When 10,000 users simultaneously request a cache-miss item, only one origin fetch is made. The other 9,999 wait in the waiting list, resolved when the fetch completes
Config propagation	Replicated Log + Gossip Dissemination	CDN routing rules, cache policies, and SSL certificates are replicated via a log to all PoPs, then gossiped within each PoP cluster
Bandwidth optimization	Request Batch	Edge→origin requests for small objects are batched (HTTP/2 multiplexing, range requests) to reduce per-request overhead on the origin link

KEY INSIGHT	CDN is essentially a globally distributed cache with eventual consistency. The Request Waiting List pattern (collapse-on-miss) is perhaps its most important optimization — without it, a cache miss causes a thundering herd to the origin.

Sequencer (ID Generator)

Infrastructure

OVERVIEW	Generates globally unique, roughly time-ordered IDs at high throughput (Twitter Snowflake, UUID v7, Mongo ObjectID). Critical for distributed systems that need ordering without a single bottleneck.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Monotonic ordering	Lamport Clock / Hybrid Clock	Each ID embeds a logical or hybrid timestamp to ensure IDs generated later sort after earlier ones — even across nodes. HLC is preferable for real-time ordering
Node coordination	Leader and Followers + Generation Clock	If one sequencer node fails, a new leader is elected (generation clock increments). The new leader's IDs are distinguished by the higher generation in the ID's node field
Uniqueness per node	Fixed Partitions	Each sequencer node owns a fixed shard of the ID space (e.g., 10-bit machine ID in Snowflake). No coordination needed for uniqueness — partitioned by node ID
Clock skew handling	Clock-Bound Wait	If the wall clock goes backward (NTP adjustment), the sequencer waits until the clock catches up before issuing new IDs — prevents duplicate IDs from clock regression
High throughput	Singular Update Queue + Request Batch	ID requests are processed by a single-threaded queue to serialize increments without locks. Batches of IDs are pre-allocated to avoid per-request disk I/O

KEY INSIGHT	The Hybrid Clock is the soul of a sequencer — it provides real-time proximity, monotonicity, and causal ordering in one number. Clock-Bound Wait handles the edge case that breaks naive timestamp-based IDs.

Distributed Monitoring

Infrastructure

OVERVIEW	Collects metrics, traces, and logs from thousands of nodes. Must handle high ingest throughput, long retention, and fast query — while itself being highly available.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Metrics ingest pipeline	Write-Ahead Log + Segmented Log	Every metric data point written to ingest nodes goes through a WAL (Kafka topic). Segmented log enables efficient time-based retention and deletion
Metric propagation	Replicated Log + Follower Reads	Metric data is replicated across ingest nodes. Queries can be served from followers (eventually consistent is fine for dashboards — you don't need the last 50ms of data)
Node health	HeartBeat + Gossip Dissemination	Monitoring agents heartbeat the collection tier. Agent-to-agent gossip (like Prometheus's federation or Consul health) propagates node availability
Alert state	Consistent Core + State Watch	Alert rules and their firing state are stored in a consistent core. State watches trigger on threshold crossings, enabling alertmanager-style deduplication and silencing
Time-series ordering	Hybrid Clock + Versioned Value	Each metric sample is timestamped with HLC. Out-of-order samples (network delay) are stored as versioned values and merged during compaction
Scrape coordination	Lease	In Prometheus HA pairs, a lease ensures only one instance scrapes each target — preventing duplicate data ingestion and double-counting in alerts
Long-term storage	Low-Water Mark + Key-Range Partitions	Retention policy uses low-water mark to determine which time-range segments to delete. Data is key-range partitioned by (metric_name, timestamp) for efficient time-range queries

KEY INSIGHT	Monitoring is a write-heavy, time-ordered system. The Segmented Log + Key-Range Partitions combination is critical — metrics are naturally time-series data and must be partitioned by time for efficient range queries and retention.

Monitor Server Side Errors

Infrastructure

OVERVIEW	Captures, aggregates, deduplicates, and alerts on server-side errors (5xx, exceptions, panics) in real time across a fleet of services.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Error event ingest	Write-Ahead Log + Request Batch	Error events are batched at the agent level before shipping (reduces ingest load). WAL on ingest nodes ensures no errors lost on crash
Deduplication	Idempotent Receiver + Versioned Value	Error events carry a content hash as ID. Idempotent receiver deduplicates bursts of the same error. Versioned values track occurrence counts over time
Alert deduplication	Consistent Core + Lease	Alert firing state lives in a consistent core. A lease ensures only one alertmanager instance sends the PagerDuty/Slack notification even in HA setups
Error grouping/fingerprinting	Singular Update Queue	Stack trace normalization and grouping is done serially per service to avoid race conditions on counter updates (like Sentry's grouping algorithm)
Threshold tracking	State Watch	When an error rate crosses a threshold, a state watch fires the alert. Watches also trigger auto-remediation (restart a pod, drain traffic)

KEY INSIGHT	The hardest part of server-error monitoring is deduplication under high cardinality. Idempotent Receiver + Consistent Core together prevent duplicate alerts when the same error fires from 1,000 pods simultaneously.

Monitor Client Side Errors

Infrastructure

OVERVIEW	Captures JavaScript errors, mobile crashes, and performance metrics from end-user devices. Volumes are enormous (millions of clients), network is unreliable, and data must survive client offline periods.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Client-side buffering	Write-Ahead Log (client-side)	The SDK writes events to localStorage/IndexedDB (a client-side WAL) before transmitting. On reconnect, the buffer is replayed — surviving page reloads and network drops
Ingest deduplication	Idempotent Receiver	Clients retry failed uploads with the same session+sequence ID. Ingest servers deduplicate so a flaky mobile connection doesn't produce duplicate crash reports
High-volume ingest	Request Batch + Request Pipeline	SDK batches events into payloads (e.g., 50 events per POST). Ingest tier pipelines batches to Kafka without waiting for each ack — critical at millions-per-second scale
Session ordering	Lamport Clock	Events within a session carry Lamport timestamps to reconstruct the exact sequence even when events arrive out of order at the ingest tier
Sampling coordination	Consistent Core + Lease	Sampling rate decisions ("capture 1% of sessions") are stored in consistent core. All SDK instances read the same rate — a lease on the rate-change operation prevents races
Aggregation	Segmented Log + Low-Water Mark	Raw events land in a segmented log (Kafka). Aggregation jobs consume from the log; low-water mark tracks which partitions are fully processed and safe to compact

KEY INSIGHT	Client-side monitoring is unique: the WAL lives on the user's device. The client SDK implements Write-Ahead Log + Idempotent Receiver patterns in JavaScript/Swift/Kotlin to handle the hostile client network environment.

Distributed Cache

Infrastructure

OVERVIEW	A fast, in-memory cache layer (Redis, Memcached) sitting in front of your database. Must handle eviction, replication, partitioning, and consistency with the source of truth.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Data partitioning	Fixed Partitions	Redis Cluster's 16,384 hash slots are fixed partitions. Keys are mapped to slots, slots to nodes. Adding nodes only reassigns slots — no rehashing of all keys
Replication	Leader and Followers + Replicated Log	Redis Sentinel/Cluster: each shard has a primary (leader) and replicas (followers). Writes go to primary; replication log propagates to replicas async
Primary failover	HeartBeat + Generation Clock + Majority Quorum	Sentinels heartbeat Redis masters. On failure, sentinels elect a new primary using quorum (majority of sentinels must agree). Generation clock (config epoch) prevents stale primaries
Eviction ordering	Versioned Value (LRU clock)	LRU eviction uses a logical clock per key (last-access version). Approximated LRU in Redis samples N keys and evicts the one with the oldest clock — the Versioned Value pattern
Cache invalidation	State Watch	On a cache miss or TTL expiry, clients can register a watch on the backing store key. When the DB record changes, the watch fires and the cache entry is proactively invalidated
Write coalescing	Singular Update Queue + Request Batch	High-write scenarios use write-behind caching: updates queue in a singular update queue and are flushed to the DB in batches — dramatically reducing DB write load
Client deduplication	Idempotent Receiver	Cache SET operations carry idempotency keys so that client retries on a flaky network don't overwrite a fresher value written by another client between retries

KEY INSIGHT	Fixed Partitions + Generation Clock is the combination that prevents the split-brain scenario in Redis Cluster. Without the generation clock (config epoch), two nodes could both believe they are the primary for a slot.

Distributed Messaging Queue

Infrastructure

OVERVIEW	Durable, ordered message delivery between producers and consumers (SQS, RabbitMQ, Kafka). The core challenge is durability + exactly-once or at-least-once delivery guarantees.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Message durability	Write-Ahead Log + Segmented Log	Every enqueued message is written to a WAL before ACK to producer. Segmented log enables efficient retention and consumer offset tracking (Kafka's core design)
Replication	Replicated Log + Leader and Followers + High-Water Mark	Each queue partition has a leader broker and follower brokers. High-water mark = last message replicated to all in-sync replicas — consumers only see up to HWM
Consumer offset tracking	Low-Water Mark + Versioned Value	Consumer group offsets are the low-water mark — they tell the broker what's safe to delete. Offsets are versioned so concurrent consumer group updates are detected
Producer deduplication	Idempotent Receiver + Generation Clock	Kafka's idempotent producer: each producer gets a Producer ID (generation) + sequence number. Broker deduplicates retries within the same producer session
Consumer group coordination	Consistent Core + Lease	Consumer group leader election happens via the Group Coordinator (Kafka's consistent core). Each consumer holds a lease on its assigned partitions; rebalance on expiry
Partition assignment	Fixed Partitions	Queue is split into fixed N partitions. Producers hash to partitions; consumers are assigned fixed partition subsets. Adding consumers triggers rebalance, not repartition
Delivery ordering	Single-Socket Channel + Request Pipeline	Producer→broker and broker→consumer use single persistent TCP connections with pipelining (multiple in-flight requests) for high throughput ordered delivery
Batch efficiency	Request Batch	Producers accumulate messages (linger.ms + batch.size) before sending. Broker also batches when replicating to followers. This is the primary throughput lever in Kafka

KEY INSIGHT	The High-Water Mark pattern is what separates Kafka from a naive queue. It ensures consumers never read messages that might be rolled back due to leader failure — this is how Kafka achieves its durability guarantees.

Pub-Sub System

Infrastructure

OVERVIEW	Decouples publishers from subscribers via topics. Publishers emit events; subscribers receive them filtered by topic/subscription. (Google Pub/Sub, AWS SNS/SQS, NATS)

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Message fanout	Replicated Log + Follower Reads	Topic messages are stored in a replicated log. Each subscription is a follower that reads from the log at its own pace — true decoupling of pub from sub rates
Topic partitioning	Fixed Partitions	Topics are split into fixed partitions across brokers. Ordering is preserved within a partition. Publishers with the same key always go to the same partition
Subscription state	Consistent Core + State Watch	Subscription registrations and filter configs live in a consistent core. State watches notify pub-sub brokers when subscriptions are added/removed/modified
At-least-once delivery	Idempotent Receiver + Write-Ahead Log	Messages are ACKed to subscriber only after successful processing. On subscriber crash, unACKed messages are redelivered. Idempotent receiver on the subscriber side handles duplicates
Dead letter handling	Segmented Log + Low-Water Mark	Failed messages go to a dead-letter topic (its own segmented log). Low-water mark tracks how far DLQ consumers have processed; old DLQ segments are deleted after retention
Push delivery	Single-Socket Channel + Request Pipeline	Push subscriptions maintain a persistent connection to subscriber endpoints. Pipeline allows multiple in-flight deliveries without waiting for each ACK

KEY INSIGHT	Pub-Sub is a Replicated Log with multiple independent read pointers (one per subscription). The Follower Reads pattern explains why subscribers can read at different rates without blocking each other or the publisher.

Rate Limiter

Infrastructure

OVERVIEW	Enforces request limits per user/IP/API key (token bucket, sliding window). Must work across a distributed cluster of API gateway nodes without excessive coordination overhead.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Counter storage	Versioned Value + Consistent Core	Rate limit counters live in a fast consistent store (Redis). Versioned values enable compare-and-swap operations to increment atomically without distributed locks
Distributed counter sync	Gossip Dissemination	For approximate rate limiting, each gateway node gossips its local counter to peers. The global rate is estimated without expensive quorum reads — good enough for most cases
Exact rate limiting	Majority Quorum + Consistent Core	When exact limits matter (payments, login attempts), every increment goes through a consistent core with quorum acknowledgment — slower but precise
Sliding window state	Segmented Log	Sliding window rate limiting stores recent request timestamps in a time-segmented log. Low-water mark deletes segments outside the window
Config updates	State Watch + Replicated Log	When rate limit thresholds change (e.g., raising limits for a premium customer), a replicated log propagates the change. State watches on gateway nodes apply it without restart
Client retry coordination	Idempotent Receiver + Lease	Rate-limited clients receive a Retry-After header based on lease expiry. The rate limiter grants a short lease on the next allowed request window

KEY INSIGHT	The fundamental choice in rate limiting is exact vs. approximate. Exact requires Consistent Core + Quorum (expensive). Approximate uses Gossip + local counters (cheap, slightly over/under limits). Choose based on the cost of being wrong.

Blob Store

Infrastructure

OVERVIEW	Stores large binary objects (files, images, videos) with high durability and throughput. (S3, GCS, Azure Blob). The core challenge is durability at scale and efficient large-object I/O.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Object durability	Write-Ahead Log + Replicated Log	Object metadata (location, checksum, size) is committed to a replicated log before the upload is confirmed. The actual bytes are striped across storage nodes for durability
Chunk management	Segmented Log + Low-Water Mark	Large objects are split into fixed-size chunks (segments). Garbage collection uses low-water mark: chunks not referenced by any live object version are deleted
Multi-part upload coordination	Request Waiting List + Idempotent Receiver	Multi-part uploads register each part in a waiting list. Assembly fires when all parts arrive. Idempotent receiver handles re-uploaded parts from network failures
Metadata cluster	Consistent Core + Leader and Followers	Object metadata (bucket→key→location mapping) lives in a strongly-consistent metadata cluster (like GFS master / HDFS NameNode) — the textbook Consistent Core pattern
Storage node health	HeartBeat + Gossip Dissemination	Storage nodes heartbeat the metadata cluster. Node availability gossips so the metadata cluster can reroute reads/writes around failed nodes within seconds
Versioning	Versioned Value + Generation Clock	Each PUT creates a new version of the object. Generation clock ensures version numbers are monotonically increasing even across metadata cluster leader changes
Large file streaming	Request Pipeline + Request Batch	Clients pipeline upload chunks without waiting for each ACK. Storage nodes batch small chunk writes before flushing to disk — critical for throughput on HDDs

KEY INSIGHT	GFS (the blueprint for all blob stores) is literally a Consistent Core (master) + Leader-and-Followers (chunkservers). The metadata cluster holding chunk locations IS the consistent core, and the replication pattern of chunks IS the replicated log.

Distributed Search

Infrastructure

OVERVIEW	Full-text search across distributed indexes (Elasticsearch, Solr). Shards data across nodes, replicates for availability, and federates queries across shards.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Index sharding	Key-Range Partitions + Fixed Partitions	Documents are routed to shards by document ID hash (fixed) or by time range (key-range, for log search). Key-range enables time-bounded queries to hit fewer shards
Shard replication	Leader and Followers + Replicated Log	Each shard has a primary and replica shards. Writes go to primary; replicated log syncs to replicas. Elasticsearch's translog IS the write-ahead log
Cluster state	Consistent Core + Gossip Dissemination	Master nodes (Elasticsearch's elected master) maintain cluster state (shard allocation, node membership) — the Consistent Core. Cluster state changes gossip to all data nodes
Query federation	Follower Reads + Request Pipeline	Scatter-gather: coordinator sends query to all shard primaries (or replicas for read load balancing), pipelines the requests, collects results, merges
Segment merging	Segmented Log + Low-Water Mark	Lucene segments are exactly the Segmented Log pattern. Low-water mark (checkpoint) tracks which segments are fully replicated and safe to merge/delete
Index versioning	Versioned Value + Generation Clock	Index aliases with versioned backing indexes enable zero-downtime reindex: write to new index version, atomically swap alias. Generation clock ensures alias version monotonicity

KEY INSIGHT	Elasticsearch's translog (WAL) + segment files (Segmented Log) + primary/replica shards (Leader and Followers) + elected master (Consistent Core) is a direct, explicit implementation of four core patterns working together.

Distributed Logging

Infrastructure

OVERVIEW	Collects, transports, stores, and makes searchable the log output of every service in a cluster. (ELK stack, Loki, Datadog Logs). Must handle enormous write throughput and flexible querying.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Log ingestion pipeline	Write-Ahead Log + Segmented Log	Log shippers write to a WAL (Kafka) before forwarding. Segmented log is how Kafka stores log data — exact same pattern as the logging system's storage
Log ordering within a service	Lamport Clock / Hybrid Clock	Log lines from a service embed HLC timestamps. When logs from multiple replicas of a service are merged, HLC ordering reconstructs the true event sequence
Log shipping reliability	Idempotent Receiver + Request Batch	Log shippers (Fluentd, Logstash) batch log lines and retry on failure. Ingest nodes deduplicate using log line hash + timestamp as idempotency key
Index partitioning	Key-Range Partitions	Logs are partitioned by time range (today's index, yesterday's index in Elasticsearch). Key-range partitioning lets queries that filter by time range skip old shards entirely
Tail & stream	State Watch + Follower Reads	"tail -f" functionality: clients register a state watch on the latest log segment offset. Reads come from followers for scalability — eventual consistency acceptable for log tailing
Retention	Low-Water Mark + Segmented Log	Retention policy configures a low-water mark (e.g., 30 days). Log segments older than the mark are deleted. Per-topic/per-stream retention is independently configured

KEY INSIGHT	Distributed logging IS a distributed messaging queue (Kafka) feeding a distributed search system (Elasticsearch). Every pattern from both those systems applies here — logging is their composition.

Distributed Task Scheduler

Infrastructure

OVERVIEW	Schedules and executes tasks (cron jobs, async jobs, DAG workflows) reliably across a cluster. (Celery, Temporal, Airflow). Must handle worker failures, duplicate execution, and ordered dependencies.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Task assignment	Consistent Core + Lease	Task assignments are leased to workers. If a worker crashes, its lease expires and the task is reassigned. etcd/ZooKeeper is the consistent core holding the assignment state
Exactly-once execution	Idempotent Receiver + Generation Clock	Each task has a generation (attempt number). Workers must register their generation before executing. Stale workers (lower generation) are rejected — prevents duplicate execution
Task queue	Write-Ahead Log + Replicated Log	Pending tasks are stored in a replicated WAL. Leader assigns tasks; followers are hot standbys. The WAL ensures tasks survive scheduler node failures
DAG ordering	Singular Update Queue + Request Waiting List	DAG dependency resolution uses a singular update queue (one thread advances the DAG state). Task completion triggers waiting downstream tasks via the waiting list
Worker health	HeartBeat + State Watch	Workers heartbeat the scheduler. On heartbeat miss, a state watch fires the task reassignment logic — immediate reaction without polling
Cron scheduling	Hybrid Clock + Versioned Value	Cron triggers use HLC timestamps to ensure a job fires exactly once even if the scheduler fails mid-second. Versioned values record the last-fired time to prevent double-firing

KEY INSIGHT	The Lease pattern is the entire foundation of task scheduling correctness. Without leases, a slow worker and its replacement could both execute the same task simultaneously. Lease expiry is the fencing mechanism.

Sharded Counter

Infrastructure

OVERVIEW	Counts events at extreme throughput (likes, views, clicks) without a single counter bottleneck. The challenge is combining high write throughput with eventually-correct read aggregation.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Write distribution	Fixed Partitions	Counter is split into N shards (fixed partitions). Each write goes to a random shard. Reads aggregate all shards. N shards = N× write throughput with no coordination
Shard replication	Leader and Followers + Replicated Log	Each shard is replicated for durability. Write to the shard's leader; replicated log propagates to followers. Loss of a shard = loss of that partition's count
Periodic aggregation	Low-Water Mark + Segmented Log	Raw increment events land in a segmented log (Kafka). Aggregation jobs run periodically; low-water mark tracks which events are already aggregated and safe to discard
Read consistency	Versioned Value + Follower Reads	Aggregated counts are stored as versioned values. Reads can come from followers (stale by seconds — acceptable for like counts). Version is the aggregation timestamp
Conflict-free merging	Version Vector	In multi-region sharded counters, each region maintains its own version vector. Merge = sum of all region vectors. No coordination needed for the merge — CRDTs implement this
Overflow detection	State Watch + Consistent Core	State watch on a shard's count fires when approaching overflow (e.g., INT_MAX) and triggers shard splitting or promotion to a larger type

KEY INSIGHT	Sharded Counter is the purest example of Fixed Partitions applied to a single logical value. The Version Vector/CRDT angle makes it the textbook use case for conflict-free distributed counters — no coordination on the write path.

YouTube / TikTok (Video Platform)

Product Systems

OVERVIEW	Massive video ingestion, transcoding, global delivery, and recommendation at billion-user scale. Videos must be durable immediately after upload; playback must handle global spikes on viral content.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Video upload pipeline	Write-Ahead Log + Idempotent Receiver	Upload server writes incoming video chunks to WAL before ACK. Client retries with same upload session ID — idempotent receiver prevents duplicate chunks
Video metadata	Versioned Value + Replicated Log	Video title, description, status (processing/live/deleted) are versioned. Status transitions replicated to all regions. Versioning detects stale caches showing 'processing' for live videos
Transcoding coordination	Consistent Core + Lease	Only one transcoding worker should process a video. Lease granted to the worker; if transcoding pod crashes, lease expires and another worker picks up from the last checkpoint
CDN delivery	Segmented Log + Fixed Partitions	Video is chunked into DASH/HLS segments (the Segmented Log pattern — literally). Each segment is independently cached at CDN edges. Fixed partition = consistent edge routing
View count	Sharded Counter pattern → Version Vector	View counts use sharded counters with version vectors for multi-region merging. Eventual consistency acceptable — nobody cares if views show 1.2M vs 1.200001M
Viral spike handling	Request Batch + Follower Reads + CDN	During viral events, reads scatter across CDN (follower reads at the edge). Origin sees batched requests from CDN shields, not individual viewer requests
Recommendation feed	Gossip Dissemination + State Watch	Model updates propagate to recommendation servers via gossip. State watches notify servers to reload the model — avoids synchronized restart storms
Comment system	Replicated Log + Key-Range Partitions	Comments are stored in a replicated log partitioned by video ID (fixed) and time (key-range within video). Enables efficient recent-comment queries

KEY INSIGHT	The Segmented Log pattern appears physically in HLS/DASH video chunking. Each TS segment in an M3U8 playlist IS a log segment. CDN delivery of video is literally a distributed segmented log serving system.

Ticketmaster (Event Ticketing)

Product Systems

OVERVIEW	Sells tickets to high-demand events where inventory is strictly limited and overselling means disaster. Flash sales create extreme write contention on the same rows. This is the hardest consistency problem in consumer software.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Seat inventory	Consistent Core + Majority Quorum	Seat availability MUST be strongly consistent. Uses a consistent core (etcd/ZooKeeper or serializable DB) with quorum writes. No eventual consistency — overselling is unacceptable
Seat reservation (hold)	Lease	When a user selects seats, a short-duration lease (5-10 min) is granted. Seat is held but not sold. On payment, lease is converted to a permanent sale. On timeout, seat returns to inventory
Flash sale queue	Request Waiting List + Singular Update Queue	During on-sale, millions of requests arrive simultaneously. A waiting list queues them; a singular update queue serializes seat assignments one by one — no concurrent updates to the same seat
Payment idempotency	Idempotent Receiver + Write-Ahead Log	Payment charge requests carry idempotency keys. On payment provider timeout, retry is safe — idempotent receiver prevents double charges. WAL logs the intent before charging
Inventory replication	Replicated Log + High-Water Mark	Inventory state replicates to read replicas for scale. High-water mark ensures users see committed sales only — no ghost seats showing as available during replication lag
Distributed locking	Consistent Core + Generation Clock	Venue-level locks use the consistent core with generation clocks. If the lock-holder crashes and restarts, its old generation is rejected — prevents double-assignment
Queue position	Lamport Clock + Request Waiting List	Virtual queue assigns Lamport-ordered positions. Users see "you are #4,521 in queue" — positions assigned at entry, served in order

KEY INSIGHT	Ticketmaster is the canonical use case for Lease + Consistent Core. The seat hold lease is the mechanism that prevents inventory appearing oversold during the payment window. Without it, two users could both complete checkout for the same seat.

DoorDash / Uber Eats (Food Delivery)

Product Systems

OVERVIEW	Matches customers, restaurants, and delivery drivers in real time. State machines are complex: order → confirmed → preparing → picked up → delivered. Location data is high-frequency. ETAs must be real-time.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Order state machine	Replicated Log + Consistent Core	Order states transition via events in a replicated log. Consistent core ensures state machine transitions are serialized (no order going from 'delivered' back to 'preparing')
Driver location	Gossip Dissemination + Versioned Value	Driver GPS pings arrive every 4 seconds. Gossip propagates location to nearby dispatch nodes. Versioned value: only the latest location matters; old locations are discarded
Driver assignment	Consistent Core + Lease	Driver assignment uses a lease: driver D is assigned to order O. If driver app crashes, lease expires, re-assignment happens. Prevents two orders assigning to the same driver
ETA calculation	Follower Reads + Hybrid Clock	ETA reads from follower replicas of traffic/routing data — staleness of seconds is acceptable. HLC timestamps on map updates enable freshness reasoning
Restaurant availability	State Watch + HeartBeat	Restaurant tablet heartbeats to signal it's online. State watch fires when a restaurant goes offline mid-order — triggers re-routing or customer notification
Payment	Idempotent Receiver + Write-Ahead Log	Charge at order placement is idempotent (retried safely). WAL ensures charge intent survives a payment service crash between charge and confirmation
Batch dispatch optimization	Request Batch + Singular Update Queue	Dispatch algorithm batches nearby order-driver pairs and optimizes assignments in bulk (one solver run per N ms) rather than greedily assigning one by one

KEY INSIGHT	Driver assignment using Lease is the key correctness pattern. The lease duration should be just long enough for the driver to accept — too long and re-assignment is slow after a crash; too short and spurious re-assignments happen on slow networks.

Quora / Reddit (Q&A / Forum)

Product Systems

OVERVIEW	Community content platform with voting, comments, and feed ranking. Read-heavy with bursty writes on viral content. Consistency on votes matters less than on financial transactions.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Post/question storage	Replicated Log + Follower Reads	Post content replicates to read replicas; queries served from followers. Eventual consistency (seconds of lag) is fine — nobody notices if upvote count lags by 2 seconds
Vote counting	Sharded Counter → Version Vector	Upvote/downvote counts use sharded counters. Multi-datacenter: version vectors merge regional counts. Acceptable if counts are eventually consistent across DCs
Feed ranking	Versioned Value + Segmented Log	Feed ranking snapshots are versioned. Each new feed generation is a new version. Segmented log stores historical ranking events for audit and ML training
Comment tree	Key-Range Partitions + Replicated Log	Comments partitioned by post ID. Within a post, key-range by parent_comment_id enables efficient subtree queries. Replicated for availability
Search	Follower Reads + Key-Range Partitions	Full-text search reads from search replicas (follower reads). Indexed by post creation time (key-range) for efficient recent-post queries
Notification fanout	Replicated Log + State Watch	When a post receives a reply, a replicated log event triggers notification fanout. State watches on follower subscriptions deliver notifications to active sessions
Spam/abuse detection	Singular Update Queue	Abuse detection pipeline runs in a singular update queue — serial processing prevents race conditions on user ban/score-update state

KEY INSIGHT	Reddit/Quora's vote counter is the textbook sharded counter use case. The insight is that exact vote counts at any given millisecond don't matter — eventual consistency is both sufficient and dramatically cheaper.

Google Maps (Mapping Platform)

Product Systems

OVERVIEW	Serves map tiles, geocoding, routing, and real-time traffic to billions of users. Map data is enormous (petabytes), routing requires low-latency graph traversal, and traffic data changes every minute.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Map tile storage	Key-Range Partitions + Segmented Log	Map tiles are keyed by (zoom_level, tile_x, tile_y) — a natural key-range partition. Tile updates are stored in a segmented log; new tile versions computed from vector data changes
Tile CDN delivery	Follower Reads + Versioned Value	Tiles cached at CDN edges (follower reads). Each tile is versioned (map version + tile coordinates). ETags implement Versioned Value pattern for HTTP cache validation
Real-time traffic	Gossip Dissemination + Hybrid Clock	Probe data from millions of phones gossips through edge aggregators. HLC timestamps enable freshness-based merging — only the most recent probe for a road segment counts
Route computation	Consistent Core + Follower Reads	Graph weights (traffic data) live in a consistent core updated every minute. Route servers read from followers (slightly stale traffic OK for routing — 60s lag is fine)
Map data updates	Replicated Log + Write-Ahead Log	When a road changes, the update is committed to a WAL and replicated log. All tile-rendering servers receive the update and regenerate affected tiles
Places/POI data	Versioned Value + Key-Range Partitions	Business data (hours, reviews, photos) is versioned. Partitioned by geohash (a key-range partition in 2D space) for efficient "POIs near me" queries
ETA prediction	Segmented Log + Follower Reads	Historical travel times stored in a segmented time-series log. ML models read from followers for training. Real-time ETA reads from latest traffic followers

KEY INSIGHT	Geohash partitioning IS the Key-Range Partitions pattern applied to 2D space. A geohash is a Z-curve encoding that maps 2D coordinates to a 1D key, making all 2D range queries (bounding box lookups) into key-range queries.

Discord (Real-Time Chat)

Product Systems

OVERVIEW	Real-time messaging at massive scale: billions of messages, millions of concurrent WebSocket connections, and sub-100ms message delivery. Voice and video add real-time media complexity.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Message storage	Write-Ahead Log + Key-Range Partitions	Messages written to WAL (Cassandra, ScyllaDB). Partitioned by (channel_id, bucket_time) — key-range for efficient recent message queries. Discord famously migrated this to ScyllaDB
Message ordering	Snowflake IDs (Hybrid Clock)	Discord message IDs are Snowflake IDs — time + node + sequence. This IS the Hybrid Clock pattern: monotonic, real-time, globally unique. Sort by ID = sort by time
Real-time delivery	Single-Socket Channel + State Watch	Each connected client has a single WebSocket (single-socket channel). Message fan-out uses state watches: new message event triggers push to all channel member connections
Server membership	Gossip Dissemination + HeartBeat	Guild/server membership propagates via gossip across Discord's node cluster. Node health tracked via heartbeats — enables routing WebSocket connections to healthy nodes
Presence (online/offline)	Gossip Dissemination + Lease	Presence state (online/idle/DND) gossips across the cluster. A lease on each presence token means if the client disconnects, presence expires within the lease window
Read state (unread badges)	Versioned Value + Follower Reads	Last-read message ID per channel is a versioned value per user. Stored in a fast key-value store, read from replicas (followers) — slightly stale unread counts are fine
Message search	Follower Reads + Key-Range Partitions	Full-text search index reads from followers. Messages indexed by (server_id, timestamp) for efficient server-scoped, time-bounded search queries

KEY INSIGHT	Discord's message ID IS a Hybrid Clock timestamp, and their message storage partitioning IS the Key-Range partition pattern. When Discord wrote about their Cassandra→ScyllaDB migration, they were describing the operational reality of maintaining a segmented log at scale.

Proximity Service / Nearby Friends

Product Systems

OVERVIEW	Finds entities (businesses, friends, drivers) near a given location in real time. Used in Yelp, Snapchat Snap Map, Facebook Nearby Friends, Uber surge pricing.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Location storage	Key-Range Partitions (Geohash)	Locations stored keyed by geohash prefix — a spatial key-range partition. Proximity queries become range queries on the geohash key space (check current cell + 8 neighbors)
Location updates	Versioned Value + Singular Update Queue	Each entity's location is a versioned value (latest wins). Updates pass through a singular update queue per entity to serialize location state changes — no race conditions
Nearby query fanout	Follower Reads + Request Batch	Proximity queries read from geo-partitioned followers. Batch nearby queries together ("give me all friends within 500m of these 100 users") for efficiency
Real-time location streaming	Gossip Dissemination + Hybrid Clock	Friend location updates gossip through region servers. HLC timestamps determine freshness — discard location updates older than the current known location
Geofence triggers	State Watch + Consistent Core	Geofence entry/exit events use state watches on geohash cells. Consistent core holds the geofence definitions; state watch fires when an entity enters/leaves
Driver/friend visibility	Lease + HeartBeat	A friend's location is visible only while their lease is active (app in foreground). App heartbeats renew the lease. Backgrounding → lease expires → removed from nearby

KEY INSIGHT	The Geohash + Key-Range Partitions combination is the architectural foundation of every proximity service. A geohash prefix IS a key-range boundary — all entities in a geographic cell share a key prefix, making bounding-box queries trivially efficient.

Uber (Ride Hailing)

Product Systems

OVERVIEW	Real-time matching of riders and drivers with stringent latency requirements (<2 seconds to match), location tracking at millions of GPS updates per second, and surge pricing under high load.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Driver location ingestion	Write-Ahead Log + Request Batch	GPS pings batched at edge before being committed to a WAL. Individual GPS updates don't need durability but location history does (for dispute resolution)
Matching engine	Consistent Core + Lease	Driver-rider matching uses a serialized matching engine backed by a consistent core. Driver is leased to rider once matched — prevents double-booking via lease exclusivity
Supply/demand data	Gossip Dissemination + Versioned Value	Supply (available drivers) and demand (open ride requests) gossip across dispatch nodes. Versioned values ensure stale supply data doesn't affect surge pricing
Surge pricing calculation	Follower Reads + Singular Update Queue	Surge reads supply/demand from followers (slightly stale OK). Price computation runs in a singular update queue per zone — one computation at a time prevents race conditions
Trip state machine	Replicated Log + State Watch	Trip state (requested→matched→en route→arrived→completed) lives in a replicated log. State watches notify rider/driver apps of each transition
ETA	Hybrid Clock + Key-Range Partitions	Traffic data partitioned by geohash (key-range). HLC timestamps on traffic observations determine freshness for ETA computation
Payment	Idempotent Receiver + Write-Ahead Log	End-of-trip charge uses idempotency key = trip_id. WAL logs the charge intent. Safe to retry on mobile network failures — no double charges

KEY INSIGHT	Uber's matching engine IS a Consistent Core running a Singular Update Queue. The hardest constraint is that each driver can only match to one rider — this mutual exclusion at scale requires the Lease pattern as the fencing mechanism.

Twitter / X (Microblogging)

Product Systems

OVERVIEW	Short-form content at massive scale: 500M tweets/day, real-time timelines, global trending topics. The core architectural challenge is fanout: a tweet from a user with 100M followers must reach all of them fast.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Tweet storage	Write-Ahead Log + Key-Range Partitions	Tweets written to WAL; partitioned by (user_id, tweet_id). Tweet IDs are Snowflake (Hybrid Clock). Key-range enables efficient user timeline queries
Fanout (push model)	Replicated Log + Request Waiting List	On tweet creation, a fanout service reads follower list and writes to each follower's timeline cache. Waiting list coordinates the async fanout workers
Timeline cache	Versioned Value + Follower Reads	Timeline is a cached, pre-computed list of tweet IDs per user. Reads from Redis replicas (follower reads). Versioned value detects when cache is stale vs. DB
Trending topics	Sharded Counter + Gossip Dissemination	Hashtag impression counts use sharded counters. Aggregated counts gossip to trending computation nodes. Eventually consistent — trending topics update every 30 seconds
Notification delivery	Single-Socket Channel + State Watch	Push notifications delivered over persistent connections. State watches on notification events trigger delivery. Single socket per client ensures ordering
Search / real-time index	Follower Reads + Segmented Log	Real-time tweet index (Earlybird at Twitter) reads from search replicas. New tweets flow through a segmented log into the search index with ~5 second lag
Rate limiting (API)	Consistent Core + Versioned Value	API rate limits stored in consistent core (Redis). Versioned values enable atomic CAS increments. Limits read from followers for low latency

KEY INSIGHT	Twitter's fanout problem is the most famous scale challenge in consumer software. The Replicated Log + Request Waiting List combination describes the write fanout path. For celebrities (>1M followers), Twitter uses pull (Follower Reads from timeline DB) instead of push — a critical hybrid of two patterns.

Newsfeed System

Product Systems

OVERVIEW	Aggregates content from followed users/pages into a personalized, ranked feed. Used by Facebook, LinkedIn, Instagram. Core challenge: ranking millions of items per user in milliseconds.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Feed generation (push)	Replicated Log + Request Waiting List	New post fans out to follower feed stores via a replicated log event. Fanout workers wait in a request waiting list, processing in parallel across follower shards
Feed generation (pull)	Follower Reads + Versioned Value	For high-follower-count accounts, feed is generated at read time by pulling recent posts and merging. Versioned values detect which posts are newer than the user's last read
Feed cache	Fixed Partitions + Versioned Value	Each user's feed shard (fixed partition by user_id). Cache stores the ranked feed as a versioned value — on new post arrival, cache is invalidated and rebuilt
Ranking ML model	Gossip Dissemination + State Watch	Model updates gossip to ranking servers. State watches trigger hot model reload — no restart needed. Versioned model deployment with rollback support
Social graph	Consistent Core + Key-Range Partitions	Follow relationships stored in a graph DB or wide-column store. Partitioned by user_id (fixed or key-range). Consistent core ensures follow/unfollow operations are atomic
Notification of new content	State Watch + Single-Socket Channel	When followed users post, state watches on subscription objects trigger push notifications over single-socket channels (WebSocket to mobile/web client)

KEY INSIGHT	The push-vs-pull fanout decision in newsfeed IS the Leader-and-Followers vs. Follower-Reads trade-off applied at the application layer. Push (pre-materialized feeds) = writes are expensive, reads are cheap. Pull (compute at read time) = writes are cheap, reads are expensive. Hybrid (push for regular users, pull for celebrities) is the production answer.

Instagram

Product Systems

OVERVIEW	Photo/video sharing with Stories, Reels, direct messaging, and explore. Combines the challenges of blob store (media), social graph, feed, and real-time messaging.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Photo/video storage	Segmented Log + Replicated Log + Blob Store patterns	Media chunked and stored in blob store. Metadata (location, filters, tags) in replicated log. Segmented log tracks media processing pipeline stages
Story expiry	Lease + Versioned Value	Stories are granted a 24-hour lease at publish time. Expiry is enforced by the lease mechanism — no explicit delete needed. Versioned value tracks the story's current state
Explore ranking	Follower Reads + Versioned Value	Explore feed reads from ranking service replicas (follower reads). Ranked feed snapshots are versioned — clients cache by version, re-fetch only when version changes
Direct messages	Single-Socket Channel + Write-Ahead Log	DMs delivered over WebSocket (single-socket channel). WAL on messaging servers ensures messages survive server crashes before delivery confirmation
Like/comment counts	Sharded Counter + Gossip Dissemination	Counts use sharded counters, aggregated via gossip. Instagram famously uses approximate counts ("2,341 others" rather than exact) — this IS eventual consistency by design
Feed	See Newsfeed System patterns	Instagram feed uses the same push/pull hybrid fanout described in the Newsfeed System — Replicated Log for fanout, Follower Reads for celebrity content pull

KEY INSIGHT	Instagram Stories' 24-hour expiry is a Lease at the application layer. The story IS the resource; the 24-hour TTL IS the lease duration. On expiry, the story moves to the archive, not deleted — the lease model maps perfectly.

URL Shortening Service

Product Systems

OVERVIEW	Maps short URLs to long URLs at massive read scale. Writes are rare; reads are enormous. The key challenges are uniqueness of short codes and sub-millisecond redirect latency.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Short code generation	Sequencer patterns → Hybrid Clock	Short codes generated from a distributed ID (Snowflake-style). HLC provides global uniqueness + time ordering. Base62-encode the ID for the short code
URL storage	Versioned Value + Replicated Log	URL mappings stored as versioned values — enables custom URL edits without breaking existing redirects (old version = old destination). Replicated for availability
Read path (redirect)	Follower Reads + Distributed Cache	The redirect path reads from cache (Redis) backed by DB followers. Target: <1ms redirect. Follower reads at the cache layer make this achievable
Cache warming	State Watch + Replicated Log	When a URL is created, a state watch triggers cache warming across regional caches. Replicated log propagates the mapping to all regions before the first redirect
Analytics (click counting)	Sharded Counter + Segmented Log	Click counts use sharded counters. Raw click events land in a segmented log for async analytics — who clicked, when, from where — without slowing the redirect path
Custom domain routing	Consistent Core + Key-Range Partitions	Custom domain → short code mapping stored in consistent core. Domains partitioned by TLD + domain (key-range) for efficient domain-level operations

KEY INSIGHT	URL shortening is architecturally dominated by the extreme read/write asymmetry (10,000:1 reads to writes). This makes it the canonical use case for Follower Reads + aggressive caching — the write path correctness matters less than the read path speed.

Web Crawler

Product Systems

OVERVIEW	Systematically fetches and indexes web content at scale (Googlebot). Manages politeness (rate limits per domain), deduplication, link frontier, and re-crawl scheduling.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
URL frontier	Segmented Log + Key-Range Partitions	URL queue stored as a segmented log partitioned by domain (key-range). All URLs for the same domain in the same partition ensures per-domain politeness limits are enforced locally
Crawl deduplication	Idempotent Receiver + Versioned Value	Crawled URL fingerprints stored as versioned values. Before fetching, check if URL was recently crawled (idempotent: don't re-fetch if fingerprint unchanged since last crawl)
Distributed fetching	Consistent Core + Lease	URL assignments to crawler workers use leases. If a crawler dies mid-fetch, its URL lease expires and the URL returns to the frontier for reassignment
Politeness rate limiting	Rate Limiter patterns	Per-domain crawl rate uses the same patterns as the Rate Limiter system — consistent core for exact per-domain limits, lease for next-allowed-crawl time
Content storage	Write-Ahead Log + Blob Store patterns	Raw HTML written to blob store. WAL ensures every fetched page is durable before the URL is marked as crawled in the frontier
Re-crawl scheduling	Hybrid Clock + Segmented Log	Re-crawl priority computed from last-crawled HLC timestamp + content change frequency. Priority queue implemented as a time-keyed segmented log

KEY INSIGHT	The web crawler frontier IS a distributed task scheduler (Consistent Core + Lease) with Key-Range partitioning by domain. Every pattern from the Distributed Task Scheduler applies here, with Key-Range Partitions adding the politeness constraint.

WhatsApp / Facebook Messenger

Product Systems

OVERVIEW	Real-time 1:1 and group messaging with end-to-end encryption, message delivery guarantees, offline delivery, and media sharing. 100B+ messages per day.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Message delivery guarantee	Write-Ahead Log + Idempotent Receiver	Messages written to WAL before delivery attempt. Client retries with message ID on network failure. Server-side idempotent receiver prevents duplicates — delivers exactly once
Offline delivery	Replicated Log + Versioned Value	Undelivered messages stored in a replicated log (durable mailbox). Versioned by sequence number per conversation. On reconnect, client syncs from last acknowledged version
Message ordering	Lamport Clock / Hybrid Clock	Messages carry HLC timestamps. In group chats, server assigns a total order. In 1:1 chats, Lamport clocks provide causal ordering across device switches
Online presence	Gossip Dissemination + Lease	Online status gossips across messaging nodes. Lease on presence token: app in foreground = active lease renewal. Background → lease expires → 'last seen' timestamp recorded
Group membership	Consistent Core + Replicated Log	Group membership changes (add/remove members) go through consistent core for atomicity. Changes fan out via replicated log to all member devices
Read receipts	State Watch + Versioned Value	Delivery and read receipts use state watches on message IDs. Versioned value stores delivery state (sent/delivered/read) — clients watch for state transitions
Media delivery	Blob Store patterns + Request Pipeline	Photos/videos uploaded to blob store. Download URLs pipelined to recipient. Chunked download uses segmented log (HLS-style) for large media
End-to-end encryption key distribution	Consistent Core + Versioned Value	Signal Protocol key bundles stored in consistent core. Versioned — if a key is compromised, new version supersedes. Clients verify version before encrypting

KEY INSIGHT	WhatsApp's offline delivery queue IS the Replicated Log pattern applied to messaging. Each user has a durable mailbox (replicated log) that retains messages until the user acknowledges receipt. The sequence number IS the high-water mark for each conversation.

Typeahead Suggestion (Search Autocomplete)

Product Systems

OVERVIEW	Returns prefix-matched suggestions in <100ms as the user types. Must handle billions of queries, personalize results, and update with trending terms in near-real-time.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Trie / index storage	Key-Range Partitions	Trie nodes partitioned by key prefix (key-range). All terms starting with 'ap' are on the same shard. Enables prefix scan without cross-shard scatter for short prefixes
Suggestion caching	Versioned Value + Follower Reads	Popular prefix completions cached as versioned values (computed hourly). Reads from cache replicas (follower reads) for sub-millisecond response. Cache miss falls through to trie
Trending terms update	Replicated Log + State Watch	Trending query log (Kafka) feeds hourly aggregation. New trending suggestions pushed to suggestion nodes via state watch — hot reload without cache flush
Personalization	Versioned Value + Follower Reads	User query history stored as versioned values. Personalized suggestions read from per-user history replicas (follower reads). History versioned to detect freshness
Index updates	Write-Ahead Log + Singular Update Queue	New terms added to the index via WAL. Singular update queue serializes trie mutations — concurrent trie updates without a queue cause data corruption
Query logging	Segmented Log + Low-Water Mark	Every typed query logged to a segmented log for ML training. Low-water mark tracks which logs are already incorporated into the model and safe to delete

KEY INSIGHT	Typeahead is dominated by the Key-Range Partitions pattern. Partitioning by prefix is the only way to make prefix scans efficient — it collocates all completions for a given prefix on the same shard, turning a distributed scatter-gather into a local lookup.

Google Docs (Collaborative Editing)

Product Systems

OVERVIEW	Real-time collaborative document editing where multiple users can type simultaneously and see each other's changes with <100ms latency. The core challenge is merging concurrent edits without conflicts.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Operational Transform / CRDT	Version Vector + Versioned Value	Each edit operation carries a version vector (which operations it has seen). OT server merges concurrent ops using version vectors to detect and resolve conflicts
Operation log	Replicated Log + Write-Ahead Log	All edit operations written to a WAL and replicated. This IS the document's source of truth — the document state is derived by replaying the operation log
Real-time delivery	Single-Socket Channel + Request Pipeline	Each client maintains a single WebSocket to the collaboration server. Operations pipelined — client doesn't wait for server ACK before sending the next keystroke
Cursor/presence	Gossip Dissemination + State Watch	Collaborator cursors and selections gossip through the session server. State watches notify all clients when another cursor moves — drives the colored cursor UI
Document locking (section locks)	Lease + Consistent Core	When a user locks a section for exclusive edit, a lease is granted. On disconnect, lease expires and other users can edit. Consistent core holds lock state
Offline editing sync	Version Vector + Replicated Log	Offline edits accumulate locally. On reconnect, version vector determines what the server has seen; unseen local ops are replayed against the replicated log
Document storage	Segmented Log + Low-Water Mark	Document history stored as a segmented log of operations. Low-water mark = last snapshot; old operation segments before the snapshot are compressible/deletable
Autosave / checkpointing	Write-Ahead Log + Versioned Value	Autosave computes a document snapshot from the operation log WAL and stores it as a versioned value. On load, start from the latest checkpoint + replay remaining ops

KEY INSIGHT	Google Docs is the most direct application of the Version Vector pattern in consumer software. Every operation in Operational Transform carries version information equivalent to a version vector — it's how the OT server knows which concurrent operations conflict and must be merged.

Deployment System (CI/CD)

Product Systems

OVERVIEW	Manages building, testing, and deploying software to production. Must handle rollbacks, canaries, dependency ordering, and worker failures without leaving systems in a half-deployed state.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Deployment state machine	Replicated Log + Consistent Core	Deployment jobs (build/test/deploy/rollback) transition through states in a replicated log. Consistent core ensures exactly one controller drives each deployment forward
Worker assignment	Consistent Core + Lease	Build/deploy workers are assigned jobs via leases. If a worker crashes mid-build, lease expires, job is reassigned to a new worker that re-runs from the last checkpoint
Artifact storage	Write-Ahead Log + Versioned Value + Blob Store	Build artifacts stored in blob store. Each artifact version is immutable (Versioned Value). WAL records which artifact version was deployed to which environment
Canary / progressive rollout	State Watch + Follower Reads	Canary traffic percentage stored in consistent core. State watches notify load balancers to adjust routing. Follower reads of error metrics determine whether to advance or rollback
Config distribution	Replicated Log + State Watch	Environment variables and secrets replicated to all target nodes via replicated log. State watches trigger config reload without process restart
Dependency ordering (DAG)	Singular Update Queue + Request Waiting List	Build DAG dependencies resolved via request waiting list — a job waits until all its dependencies complete. Singular update queue serializes DAG state transitions
Rollback	Versioned Value + Write-Ahead Log	Rollback = deploy the previous artifact version. WAL records the exact previous version per service. Versioned artifact store makes rollback atomic and safe

KEY INSIGHT	The Lease pattern is the safety mechanism that prevents split-brain deployments. Without it, a slow worker and its replacement could simultaneously deploy different versions of a service to the same cluster. The lease ensures only one deployer is active at a time.

Payment System

Product Systems

OVERVIEW	Processes financial transactions with strict exactly-once semantics. A payment must never be lost AND never be applied twice. This is the hardest consistency problem in production systems.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Transaction atomicity	Two-Phase Commit + Write-Ahead Log	A payment (debit A, credit B) uses 2PC across accounts. WAL on coordinator records the decision. If coordinator crashes after PREPARE but before COMMIT, WAL enables recovery
Exactly-once processing	Idempotent Receiver + Generation Clock	Every payment request has an idempotency key. Generation clock on payment sessions rejects stale retry attempts. Server caches results for the deduplication window
Account balance consistency	Consistent Core + Majority Quorum	Balance reads and writes go through a strongly-consistent store with quorum acknowledgment. No eventual consistency — incorrect balances are a regulatory violation
Ledger storage	Write-Ahead Log + Replicated Log + Versioned Value	Every transaction creates a new ledger entry (append-only WAL). Replicated to multiple nodes. Each account balance is derived from its transaction log — Versioned Value via event sourcing
Fraud detection pipeline	Segmented Log + Follower Reads	Transaction events stream through a segmented log (Kafka). Fraud ML models read from followers — slightly stale is OK for fraud scoring; real-time is only for blocking
Settlement	Request Batch + Request Waiting List	End-of-day settlement batches transactions into settlement runs. Waiting list tracks which transactions are included; batch processing sends them to the payment network
Distributed lock (prevent double-spend)	Lease + Consistent Core	Before debiting, acquire a short lease on the source account. Prevents concurrent payments from the same account from both succeeding. Lease released after COMMIT

KEY INSIGHT	Two-Phase Commit exists almost exclusively for payment systems and financial transactions. This is the system where the performance cost of 2PC is justified by the correctness requirement. No other consumer system pays this price routinely — payments uniquely require it.

ChatGPT / AI Chat System

Product Systems

OVERVIEW	Real-time LLM inference with streaming responses, conversation history, and massive per-request compute. The infrastructure challenge is GPU cluster management, routing, and context management at scale.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Request routing	Consistent Core + Lease	Long-running inference requests are routed to a specific GPU worker. A lease ensures the request stays on that worker for its duration. Re-routing on GPU failure uses the same lease expiry mechanism
Conversation history	Versioned Value + Replicated Log	Conversation history stored as versioned append-only log per session. Each message is a new version. Replicated for availability — session must survive worker failures mid-conversation
Streaming response	Single-Socket Channel + Request Pipeline	Streaming tokens delivered over SSE/WebSocket (single-socket channel). Pipeline: inference generates tokens → immediately streamed to client without waiting for full completion
GPU worker health	HeartBeat + Gossip Dissemination	GPU workers heartbeat the routing layer. Worker availability gossips to all router instances — GPU failures (frequent in large clusters) are detected within 2-3 heartbeat intervals
Model versioning	Versioned Value + State Watch	Model weights are versioned (GPT-4, GPT-4-turbo, etc.). State watches notify workers of new model availability for hot loading. Old versions kept for a/b testing and rollback
Rate limiting	Rate Limiter patterns + Consistent Core	Per-user token rate limits enforced via consistent core (Redis). Critical: must be exact (not approximate) since token costs are billed — Quorum acknowledgment on every increment
Prompt caching	Versioned Value + Follower Reads	Common system prompt prefixes cached (KV cache in transformer). Versioned by prompt hash. Reads from follower GPU nodes — reduce prefix computation across users with same system prompt

KEY INSIGHT	AI inference systems expose a new use case for the Lease pattern: GPU affinity. A long inference request must stay on the same GPU worker for its context window lifetime. The lease prevents the router from reassigning the session to a different worker mid-stream.

Dropbox / Google Drive (File Sync)

Product Systems

OVERVIEW	Syncs files across multiple devices and users. Must handle concurrent edits from multiple devices (offline, online), efficient delta sync, and conflict resolution.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
File change log	Write-Ahead Log + Replicated Log	Every file change creates a WAL entry (change journal). Replicated log syncs the journal across devices. Devices sync by replaying the journal from their last-synced cursor
Sync cursor	High-Water Mark + Versioned Value	Each device tracks a sync cursor (high-water mark) — the last journal entry it has processed. Server returns only changes since the cursor. Version of cursor stored as versioned value
Conflict resolution	Version Vector	Concurrent edits from two devices use version vectors to detect conflicts. If version vectors don't dominate each other, it's a true conflict → create conflict copy (Dropbox behavior)
File deduplication	Idempotent Receiver + Versioned Value	Content-addressed storage: files stored by hash. Uploading the same file twice is idempotent — hash lookup shows it already exists. Versioned by content hash
Large file chunking	Segmented Log + Request Pipeline	Large files split into fixed chunks (Blob Store pattern). Chunks uploaded in parallel with pipelining. Only changed chunks are re-uploaded (delta sync)
Share / collaboration	Consistent Core + State Watch	Shared folder membership stored in consistent core. State watches notify all members of new files — drives real-time "someone just added a file" notifications
Offline sync queue	Write-Ahead Log (client-side) + Idempotent Receiver	Offline changes buffered in a client-side WAL. On reconnect, replay WAL to server. Idempotent receiver on server handles re-uploads from interrupted syncs

KEY INSIGHT	Dropbox's sync cursor IS the High-Water Mark pattern. Devices request 'give me all changes since cursor position N'. The server returns the next batch and a new cursor. This is exactly how Kafka consumer offsets work — and it's the same pattern.

Stock Exchange

Product Systems

OVERVIEW	Matches buy and sell orders with microsecond latency, zero tolerance for duplicate or lost orders, strict price-time priority, and regulatory auditability of every event.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Order matching engine	Singular Update Queue + Consistent Core	The matching engine is a SINGLE-THREADED singular update queue — one order processed at a time, in strict arrival order. This is non-negotiable: concurrent matching causes price violations
Order book storage	Write-Ahead Log + Replicated Log	Every order (new, cancel, fill) written to WAL before acknowledgment. Replicated to standby matching engine for instant failover. WAL is the regulatory audit trail
Order deduplication	Idempotent Receiver + Generation Clock	Order IDs carry client-id + sequence-number + session-generation. Exchange rejects duplicate orders (same ID) and stale session orders (lower generation) — prevents double-execution
Market data dissemination	Replicated Log + Gossip Dissemination	Trade confirmations and order book updates replicated to market data servers. Gossip distributes to subscribers (trading firms) — multicast semantics
Settlement	Two-Phase Commit + Write-Ahead Log	Trade settlement (transfer stocks + cash between accounts) uses 2PC. Both legs must commit atomically. WAL enables recovery if settlement system crashes between prepare and commit
Clock synchronization	Hybrid Clock + Clock-Bound Wait	Order timestamps use HLC for microsecond precision and monotonicity. Clock-bound wait ensures regulatory timestamp requirements (MiFID II requires 1μs accuracy) are met
Risk controls	State Watch + Consistent Core	Pre-trade risk limits (position limits, order size caps) stored in consistent core. State watches trigger circuit breakers when limits are approached

KEY INSIGHT	The Stock Exchange is the only consumer system that REQUIRES both Two-Phase Commit (settlement) AND Singular Update Queue (matching) in its critical path simultaneously. It's also the primary use case for Clock-Bound Wait — regulatory compliance demands provably accurate timestamps.

Distributed Email Service

Product Systems

OVERVIEW	Sends, receives, stores, and delivers email at scale (Gmail, Outlook). Must handle delivery guarantees, spam filtering, global routing, and enormous storage.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Message storage	Write-Ahead Log + Key-Range Partitions	Emails stored in append-only log partitioned by (user_id, timestamp). Key-range partition enables efficient mailbox queries (inbox, date range search). Gmail's Bigtable storage IS this
Delivery guarantee	Write-Ahead Log + Idempotent Receiver	SMTP retry protocol: sender retries until ACK. Receiving server uses idempotent receiver (message-id deduplication) — prevents duplicate delivery from SMTP retries
Mailbox replication	Replicated Log + Follower Reads	User mailboxes replicated to multiple nodes. Reads from followers (slightly stale OK — email doesn't need millisecond freshness). Writes to leader only
Spam filtering pipeline	Segmented Log + Follower Reads	All incoming email flows through a segmented log (Kafka). Spam classifiers read from followers — ML scoring doesn't need real-time data, just recent-ish features
Attachment storage	Blob Store patterns	Attachments stored in blob store with content-addressing. Idempotent upload — same file attached to multiple emails stored once, referenced many times
SMTP routing	Consistent Core + Key-Range Partitions	MX record lookup (DNS) routes to receiving cluster. Within cluster, key-range by recipient domain routes to the correct mailbox shard
Search	Follower Reads + Segmented Log	Full-text email search reads from search replicas (followers). Email index maintained as a segmented log of index updates — recent emails indexed within seconds

KEY INSIGHT	Gmail's original storage system (Bigtable) IS the Key-Range Partitions pattern. Each row key is (user_id + reverse_timestamp) — the reverse timestamp ensures recent emails sort first within a user's keyspace, making inbox queries trivially efficient as a range scan.

Real-Time Gaming Leaderboard

Product Systems

OVERVIEW	Displays player rankings updated in near-real-time during live game events. Must handle millions of score updates per second and serve rankings at low latency with high accuracy.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Score storage	Versioned Value + Consistent Core	Player scores stored as versioned values in Redis Sorted Set. Score update = new version. Consistent core ensures score updates are atomic (read-modify-write without race conditions)
Leaderboard computation	Sharded Counter + Follower Reads	Aggregate leaderboards use sharded counters per score segment. Global rank reads from sorted set replicas (follower reads) — acceptable latency trade-off for read scale
Score update pipeline	Segmented Log + Write-Ahead Log	Score events land in a WAL (Kafka). Consumers update the sorted set. WAL ensures no score is lost even if the leaderboard service crashes during peak play
Regional leaderboards	Version Vector + Fixed Partitions	Per-region leaderboards use fixed partition by region. Version vectors merge regional scores for global leaderboard — eventual consistency is fine for cross-region ranking
Real-time rank notification	State Watch + Single-Socket Channel	Players watch their rank position. State watch fires when rank changes by >N positions. Delivered over single-socket WebSocket channel for in-game display
Anti-cheat / score validation	Consistent Core + Majority Quorum	Suspicious score spikes require quorum validation across game servers before committing to the leaderboard. Quorum prevents a single compromised server from inflating scores
Leaderboard snapshots	Segmented Log + Low-Water Mark	Hourly/daily snapshots stored as segments. Low-water mark discards raw score events once incorporated into the latest snapshot — keeps storage bounded

KEY INSIGHT	Redis Sorted Set IS the Versioned Value pattern specialized for ranking. Each ZADD is a versioned update (new score = new version). ZRANK is a follower read that may return a rank that's seconds stale — which is fine for leaderboards. The Consistent Core ensures the sorted set itself doesn't corrupt under concurrent updates.

Ad Click Event Aggregation

Product Systems

OVERVIEW	Counts ad impressions and clicks in near-real-time for billing and optimization. Must handle millions of events per second with exactly-once counting (billing accuracy) and low-latency query.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Event ingestion	Write-Ahead Log + Request Batch	Click events batched at edge collectors before committing to WAL (Kafka). Batching is critical — individual click events would overwhelm the ingest tier
Deduplication (exactly-once)	Idempotent Receiver + Generation Clock	Click events carry user-session + click-id + generation. Deduplication at ingest ensures each click counted exactly once even with client retries or network duplicates
Aggregation pipeline	Segmented Log + Low-Water Mark	Kafka topics (segmented log) feed aggregation jobs (Flink/Spark Streaming). Low-water mark tracks which time windows are fully aggregated and whose raw events can be discarded
Windowed counts	Versioned Value + Hybrid Clock	Aggregate counts stored as versioned values per (ad_id, time_window). HLC timestamps enable correct window boundary assignment even with late-arriving events
Billing reconciliation	Two-Phase Commit + Write-Ahead Log	Billing records committed atomically with click count finalization. 2PC ensures count and billing record are never inconsistent. WAL enables recovery from billing service crash
Query serving	Follower Reads + Key-Range Partitions	Click count queries served from read replicas (follower reads). Partitioned by (ad_id, date) — key-range for efficient per-campaign, per-day queries
Fraud detection	Sharded Counter + Gossip Dissemination	Suspicious click-rate anomalies detected by comparing sharded counter aggregates. Gossip propagates fraud signals to all ingest nodes for real-time filtering

KEY INSIGHT	Ad click aggregation is where the Segmented Log pattern provides its most direct business value: Kafka topics ARE the audit trail. Regulators and advertisers can replay the raw click log to verify counts — the WAL is both the operational mechanism and the contractual proof.

Hotel Reservation System

Product Systems

OVERVIEW	Books hotel rooms with strict inventory constraints — a room cannot be double-booked. Must handle high read traffic (browsing) with occasional high-contention writes (popular dates at popular hotels).

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Room inventory	Consistent Core + Majority Quorum	Available room inventory MUST be consistent. Read-modify-write on inventory uses quorum — same requirement as Ticketmaster seats. No eventual consistency on the booking path
Reservation hold	Lease + Write-Ahead Log	"Room is being booked" state granted as a short lease (5-10 min). WAL records the hold. If payment times out, lease expires and room returns to inventory automatically
Availability search	Follower Reads + Key-Range Partitions	Availability search reads from replicas (follower reads). Room data partitioned by (hotel_id, date_range) — key-range for efficient date-range availability queries
Booking confirmation	Idempotent Receiver + Two-Phase Commit	Booking request carries idempotency key. Payment + inventory decrement use 2PC for atomicity. Idempotent receiver handles network retries during the checkout flow
Pricing updates	Versioned Value + State Watch + Replicated Log	Dynamic pricing stored as versioned values. Price changes flow through replicated log. State watches notify caches to invalidate stale prices — critical for revenue accuracy
Overbooking (yield management)	Sharded Counter + Consistent Core	Hotels deliberately overbook by N% using sharded counters to track net bookings. Consistent core gates the last few bookings per room type with exact quorum reads

KEY INSIGHT	Hotel reservation = Ticketmaster for rooms. The Lease (hold) + Idempotent Receiver (retry-safe booking) + Two-Phase Commit (payment + inventory) trinity is the exact combination required for any inventory-constrained booking system.

Uber Eats (Food Delivery Platform)

Product Systems

OVERVIEW	Shares core architecture with DoorDash but adds restaurant discovery, menu management at scale, and loyalty/promotion systems on top of the real-time delivery coordination engine.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Restaurant menu sync	Replicated Log + State Watch	Menu changes (price updates, item availability) replicate via log to all regional clusters. State watches notify the app in real time — item going out of stock updates instantly
Order routing to restaurant	Consistent Core + Lease	Restaurant tablet is leased the order for confirmation. If tablet is unresponsive, lease expires and order is re-routed or customer is notified
Promo/coupon validation	Idempotent Receiver + Consistent Core	Coupon redemption is idempotent (same coupon code = same discount, not double discount). Consistent core enforces single-use coupons under concurrent checkout attempts
Delivery zone management	Key-Range Partitions (Geohash)	Delivery zones are geohash key-range partitions. Zone assignments to delivery drivers use the same geo-partitioned store as Uber's driver matching
Real-time order tracking	Single-Socket Channel + State Watch	Customer tracks delivery on a map. State watches fire on driver location updates. Delivered over WebSocket (single-socket channel) for real-time map rendering
Surge pricing	Follower Reads + Sharded Counter	Supply/demand data read from followers (slightly stale OK). Order volume counted via sharded counters per zone. Surge multiplier computed from ratio

KEY INSIGHT	Uber Eats vs. DoorDash: the distinguishing pattern is menu management. Menus are structured data with real-time availability changes (item sold out mid-service) — this requires State Watch on menu items, which pure delivery platforms don't need.

Facebook Messenger (Messaging Platform)

Product Systems

OVERVIEW	Messaging at Meta scale: billions of users, cross-platform (web, iOS, Android), tight integration with Facebook social graph, and group chats with thousands of members. Distinct from WhatsApp in that it is not end-to-end encrypted by default.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Message fanout (group chats)	Replicated Log + Request Waiting List	Group messages fan out to all members via a replicated log. Large groups (1000+ members) use a waiting list to coordinate async delivery workers across member shards
Inbox storage	Key-Range Partitions + Write-Ahead Log	Each user's inbox partitioned by (user_id, thread_id, timestamp) — key-range for efficient conversation list and thread message queries. WAL for durability
Cross-platform sync	High-Water Mark + Versioned Value	Message read state and delivery receipts versioned per device. Each device tracks its sync cursor (high-water mark). On new device login, sync from cursor to latest
Social graph integration	Consistent Core + Follower Reads	Friend/contact list reads from social graph replicas (follower reads). Friend data is eventually consistent — seconds of lag in seeing new friends is acceptable
Online presence at scale	Gossip Dissemination + Lease	Presence for billions of users cannot use a central store. Gossip propagates online status across presence servers. Leases ensure presence expires when app backgrounds
Notification fanout	Replicated Log + State Watch	Push notifications triggered by state watches on message delivery events. Replicated log ensures notifications are not lost even if the push notification service has a blip

KEY INSIGHT	Facebook Messenger's group chat fanout is the most scale-demanding version of the Replicated Log + Request Waiting List combination. A message sent to a 1000-person group must be durably delivered to 1000 different inboxes — the waiting list coordinates the fanout workers without blocking the sender.

Data Infrastructure System (Data Platform)

Product Systems

OVERVIEW	The internal platform for data ingestion, processing, storage, and serving: data lakes, warehouses, stream processing, and ML feature stores. Powers analytics and ML across the organization.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Data lake ingestion	Write-Ahead Log + Segmented Log	All data lands in an immutable append-only log (S3 + Kafka). Segmented by time. The lake IS the segmented log — new data is always new segments, never mutations to old ones
Stream processing	Replicated Log + Low-Water Mark	Flink/Spark Streaming reads from Kafka (replicated log). Checkpoints use low-water mark to track which offsets are fully processed — enables restart from last checkpoint after failure
Batch processing (ETL)	Segmented Log + Key-Range Partitions	ETL jobs read time-partitioned data (key-range by date). Process in segments. Low-water mark tracks which date partitions are fully ingested into the warehouse
Feature store	Versioned Value + Follower Reads	ML features stored as versioned values per entity. Training reads historical versions (point-in-time correctness). Serving reads from feature store replicas (follower reads for low latency)
Catalog / metadata	Consistent Core + Replicated Log	Table schemas, partition locations, and lineage stored in a consistent metadata store (Hive Metastore, AWS Glue). Schema changes replicated to all query engines
Data quality monitoring	State Watch + HeartBeat	Pipeline health: each pipeline stage heartbeats. State watches trigger alerts on SLA breaches (data not arriving within expected window)
GDPR/deletion	Low-Water Mark + Versioned Value	User deletion requests create a tombstone versioned value. Low-water mark: once a deletion propagates to all derived datasets, the original raw data can be purged from the lake

KEY INSIGHT	The Data Lake IS a Segmented Log with object storage (S3/GCS) as the durable backend. Partitioning by date is Key-Range Partitions. Processing is replaying the log from a checkpoint (Low-Water Mark). The entire data platform architecture reduces to these three patterns.

Chat System (Generic / Enterprise)

Product Systems

OVERVIEW	Enterprise or general-purpose chat (Slack, Microsoft Teams, generic chat APIs). Adds channels, threads, search, file sharing, integrations, and admin controls on top of core messaging.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Message persistence	Write-Ahead Log + Key-Range Partitions	Messages in a channel stored as WAL entries partitioned by (workspace_id, channel_id, timestamp). Key-range for efficient 'messages in this channel since timestamp' queries
Thread ordering	Lamport Clock + Versioned Value	Thread replies use Lamport clock for causal ordering — a reply is causally after its parent. Versioned value per thread tracks the reply tree structure
Workspace member sync	Replicated Log + State Watch	Adding/removing members from a channel replicates via log. State watches notify all channel servers to update their local membership caches immediately
Search	Follower Reads + Segmented Log	Full-text message search reads from search replicas (follower reads). Indexing pipeline reads from the channel log (segmented log) with a few seconds lag
Integration webhooks	Write-Ahead Log + Idempotent Receiver	Outbound webhooks (Slack apps) written to a WAL before delivery attempt. Webhook endpoint must be idempotent — Slack retries failed deliveries with the same event ID
Unread counts	Sharded Counter + Versioned Value	Per-user, per-channel unread count stored as versioned value. High-traffic channels use sharded counters to handle simultaneous message arrivals without counter contention
File uploads	Blob Store patterns + Single-Socket Channel	File uploads go to blob store directly. Download links delivered over persistent WebSocket (single-socket channel) to avoid polling for upload completion

KEY INSIGHT	Slack's architecture is distinct from WhatsApp in one key way: channels are shared (many readers, many writers) vs. private chats (few participants). This makes the Sharded Counter for unread counts critical — in a Slack channel with 1000 members, every message increments 1000 different user-channel unread counters simultaneously.

Nearby Friends

Product Systems

OVERVIEW	Shows which friends are near your current location in real time (Facebook Nearby Friends, Snapchat Snap Map). Combines high-frequency location updates with social graph filtering and privacy controls.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Location storage	Key-Range Partitions (Geohash) + Versioned Value	Friend locations stored by geohash prefix (key-range in 2D space). Each location is a versioned value — only the latest matters. Old locations auto-expire by version
Location update ingest	Write-Ahead Log + Request Batch	GPS pings batched at client (every 4-30 seconds depending on movement). Batch of location updates committed to WAL before fan-out to friend location stores
Friend list filtering	Consistent Core + Follower Reads	Social graph (friend relationships) lives in a consistent core. Nearby query first fetches friend list, then filters location store by friend IDs. Friend list reads from followers
Privacy controls	Consistent Core + State Watch	Privacy settings (share with everyone / friends only / no one) stored in consistent core. State watches immediately stop location sharing when privacy is changed — no polling lag
Real-time updates (push)	Single-Socket Channel + State Watch	When a friend moves into/out of proximity, a state watch fires. Notification delivered over WebSocket (single-socket channel) to update the map in real time
Geofence proximity	Key-Range Partitions + Request Waiting List	Proximity check queries nearby geohash cells (key-range range query). For push notifications, waiting list tracks which friends to notify when they enter the proximity radius

KEY INSIGHT	Nearby Friends = Proximity Service + social graph filtering. The additional pattern requirement over pure proximity service is the Consistent Core for privacy controls — location sharing must stop instantly when a user changes their privacy settings, not after a cache TTL expires.

Google Drive

Product Systems

OVERVIEW	Cloud file storage with real-time collaboration (Docs/Sheets/Slides), offline sync, sharing, and version history. Combines Dropbox-style file sync with Google Docs-style collaborative editing.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
File sync engine	Write-Ahead Log + High-Water Mark	Sync engine is identical to Dropbox: WAL records all changes, high-water mark (sync cursor) per device tracks sync state. Changes since cursor delivered on reconnect
Real-time collaboration	Version Vector + Replicated Log	For Docs/Sheets/Slides files, concurrent edits use operational transform backed by version vectors (same as Google Docs pattern). Operation log is the Replicated Log
File metadata	Consistent Core + Versioned Value	File ownership, sharing permissions, folder structure in consistent metadata store. Versioned — permission changes take effect immediately (new version supersedes old)
Sharing / access control	Consistent Core + State Watch	ACL changes propagate via consistent core. State watches notify Drive clients to refresh share state — a revoked link stops working within seconds, not after TTL
Storage deduplication	Idempotent Receiver + Versioned Value	Content-addressed blob storage (same as Dropbox). Uploading the same file from different accounts stores only one copy. Idempotent: hash lookup before write
Version history	Segmented Log + Low-Water Mark	File revision history IS the segmented operation log. Low-water mark = automatic deletion of versions older than the retention policy (30 days for basic accounts)
Offline editing	Write-Ahead Log (client) + Version Vector	Offline edits to Docs buffered in client-side WAL. On reconnect, version vectors determine which ops the server hasn't seen — only new ops are sent

KEY INSIGHT	Google Drive is architecturally the union of two systems: Dropbox (file sync via WAL + High-Water Mark) for binary files, and Google Docs (operational transform + Version Vector) for native formats. The Consistent Core for ACLs is the glue — permissions must be strongly consistent because incorrect sharing is a serious privacy violation.

Digital Wallet (e.g., PayPal, Venmo, Apple Pay)

Product Systems

OVERVIEW	Manages user balances, peer-to-peer transfers, payment processing, and transaction history. Requires the highest consistency guarantees of any consumer system — money cannot be created or lost.

Pattern Application by Component

Component / Layer	Patterns Applied	Why This Pattern Here
Balance ledger	Write-Ahead Log + Replicated Log (Event Sourcing)	Balances are NEVER stored directly — they are computed from an append-only transaction log. Every credit/debit is a WAL entry. Balance = sum of transaction log. This is event sourcing.
P2P transfer atomicity	Two-Phase Commit + Consistent Core	Transfer = debit sender + credit receiver. 2PC across both account shards. Consistent core holds the transaction coordinator state. No partial transfers under any failure mode
Double-spend prevention	Lease + Majority Quorum	Before debiting, acquire a short exclusive lease on the source balance. Quorum acknowledgment from replicas before lease is granted. Prevents concurrent sends exceeding balance
Idempotent payments	Idempotent Receiver + Generation Clock	Every payment has a client-generated idempotency key + session generation. Retries are safe — same key = same transaction, not a new charge. Generation rejects stale retries
Transaction history	Key-Range Partitions + Versioned Value	Transactions partitioned by (user_id, timestamp) for efficient statement queries (date range). Each transaction is immutable — versioned value where old versions are never modified, only new ones added
Fraud detection	Segmented Log + Follower Reads + State Watch	All transactions flow through a segmented log (Kafka). Fraud ML reads from followers. State watches trigger real-time blocks when fraud score exceeds threshold
Multi-currency / FX	Versioned Value + Hybrid Clock	Exchange rates are versioned values with HLC timestamps. Trade executions read the rate at the HLC timestamp of the transaction — prevents rate manipulation via replay attacks
Regulatory reporting	Segmented Log + Low-Water Mark	All transaction events retained in a segmented log for regulatory retention (7 years). Low-water mark marks which segments are archived to cold storage (not deleted — regulatory hold)

KEY INSIGHT	The Digital Wallet's transaction ledger IS the Write-Ahead Log used as the permanent source of truth (event sourcing). Unlike other systems where the WAL is a recovery mechanism and the 'real' data is in a database, in a wallet the WAL IS the database. Balances are computed views over the immutable log. This makes audit trivial and fraud reversal possible.

Quick Decision Cheat Sheet

Use this table when you sit down to design a new system. Match your constraint to the pattern.

Constraint / Need	Pattern(s) to Apply
I need to survive crashes without losing data	**Write-Ahead Log**
I need multiple copies of data	**Replicated Log + Leader and Followers**
I need to scale reads horizontally	**Follower Reads**
I need to avoid split-brain	**Majority Quorum + Generation Clock**
I need to detect node failures	**HeartBeat**
I need a small coordination service	**Consistent Core**
I need retries to be safe	**Idempotent Receiver**
I need to bound recovery time after failure	**Lease**
I need to react to state changes (not poll)	**State Watch**
I need to partition data by key	**Fixed Partitions (hash) or Key-Range Partitions (range queries)**
I need atomic multi-node transactions	**Two-Phase Commit**
I need ordering across nodes without a central clock	**Hybrid Clock or Lamport Clock**
I need historical / time-travel reads	**Versioned Value**
I need to detect concurrent update conflicts	**Version Vector**
I need exact timestamps for billing/compliance	**Clock-Bound Wait**
I need to count at very high write throughput	**Sharded Counter (= Fixed Partitions on a single value)**
I need to propagate info without flooding	**Gossip Dissemination**
I need to clean up old log data safely	**Segmented Log + Low-Water Mark**
I need to maximize network throughput	**Request Batch + Request Pipeline**
I need ordered messages between two nodes	**Single-Socket Channel**
I need to collect quorum responses before replying	**Request Waiting List**
I need a single-writer pattern without locks	**Singular Update Queue**
I need leader election without a voting protocol	**Emergent Leader**